We choose to work on secret feature B.

To include the “image reversal” gimmick in our gameplay, we only need to touch the drawing layer of our project. The game rules will remain the same. We will not need to rewrite bullet physics or collisions, so we can keep all the X/Y math like before.

Currently, in our project, all screens call spriteBatch.Begin(), then draw the background, entities, bullets, UI, and finally call spriteBatch.End(), and that goes straight to the back-buffer. To add the “image reversal” gimmick, we can add one more step in between. We can first dump the whole frame into a RenderTarget2D by adding a RenderTarget2D field in the Game1 class, which is our project’s main game controller. It will act as a temporary snapshot of the whole scene. RenderTarget2D is a special type of texture that we can draw into. It's derived from the Texture2D class and is essentially a 2D buffer in memory where we can render images, and then draw that image onto the screen.

Once we have that RenderTarget2D snapshot, it can be passed to an “effect” object whose only job would be to decide how to put the picture back on-screen: normal, mirrored, flipped upside-down, rotated 90°, etc. For example,

```
public interface IRenderEffect
{
   void Apply(SpriteBatch sb, RenderTarget2D source);
}
```

Concrete classes for this interface can be NoEffect, FlipXEffect, FlipYEffect, Rotate180Effect, etc and each class would work on the passed RenderTarget2D snapshot and apply the corresponding effect.

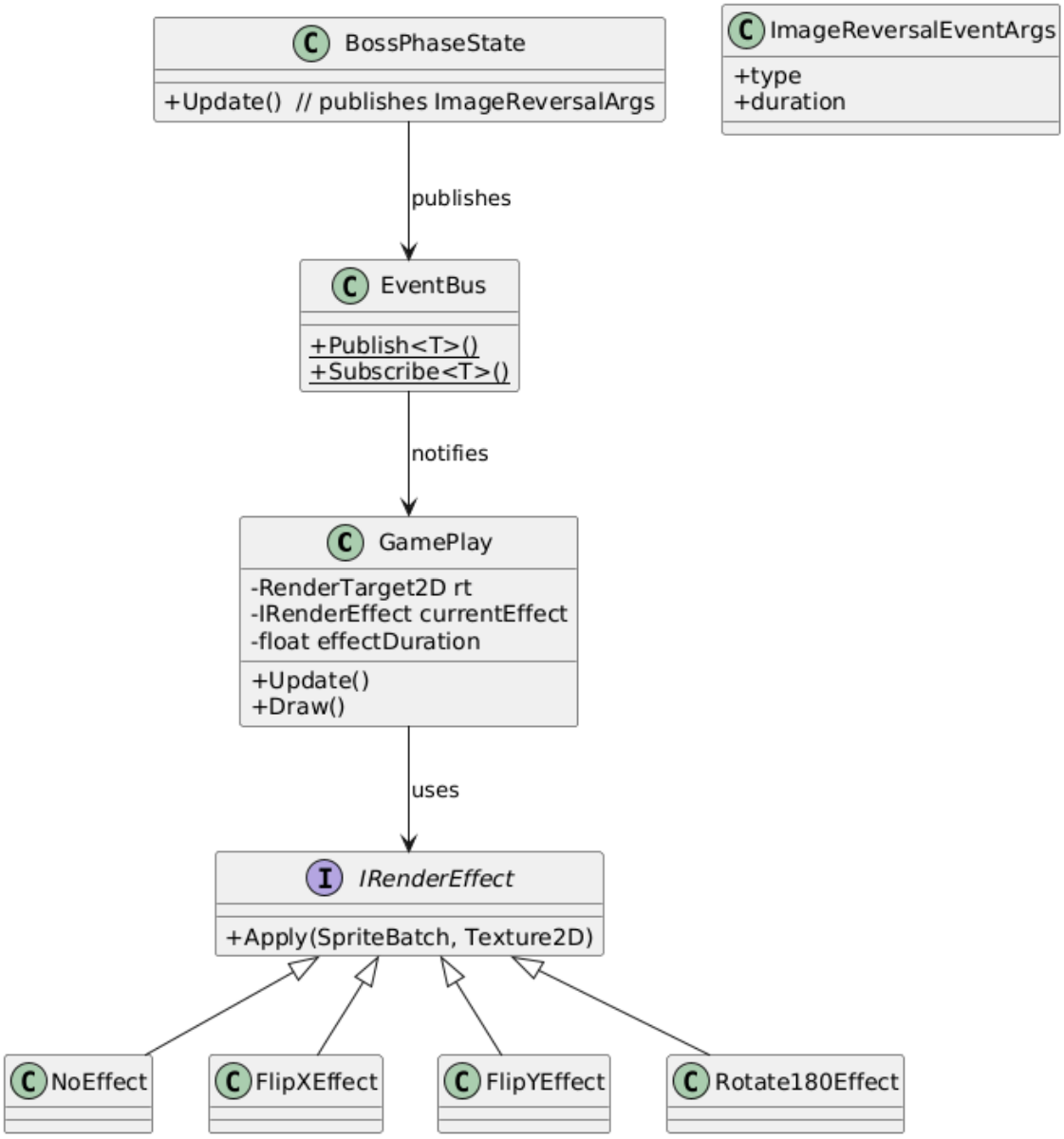
In our project, GamePlay class defines the actual battlefield screen. So, GamePlay will hold a IRenderEffect field called currentEffect and refer to it each frame. To switch effects at runtime, we can introduce a small observer class called EventBus (a static class with Publish<T>() and Subscribe<T>()) .

The new “image reversal” boss state would simply publish on the EventBus information like what kind of effect and how long it would last. Gameplay, which would be subscribing to the EventBus will then receive the event and swap in the corresponding IRenderEffect for the specified time. Later, it can revert to NoEffect.

Because the flip happens after world rendering, bullets, collisions and movement maths remain intact, and we simply draw HUD elements (health bar, menus, etc) after the effect so they are

never inverted. With this approach, every core subsystem and the patterns used remain intact in our project. Factories still produce entities without caring where the pixels end up. Movement and Fire strategies also remain the same because even after screen flip, the world coordinates remain intact, and the math that moves objects shouldn't care. All collision commands also work in the same manner. The only change would be adding a new Boss state whose only job is broadcasting the flip event.

UML Diagram for proposed changes:



Sequence Diagram: